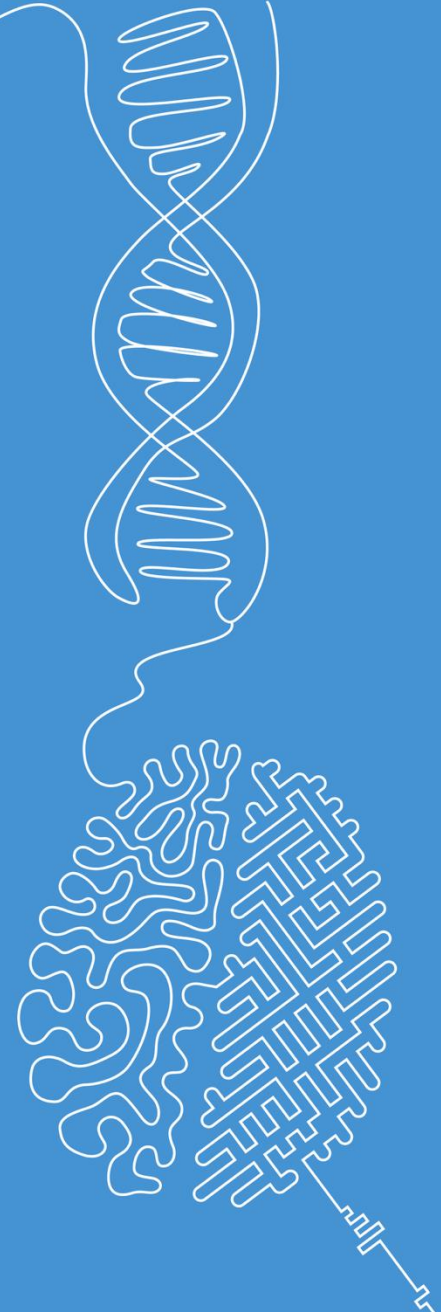


Filtering

Image and Signal Processing

Norman Juchler



Convolution

Revisited

Convolution of continuous-time signals

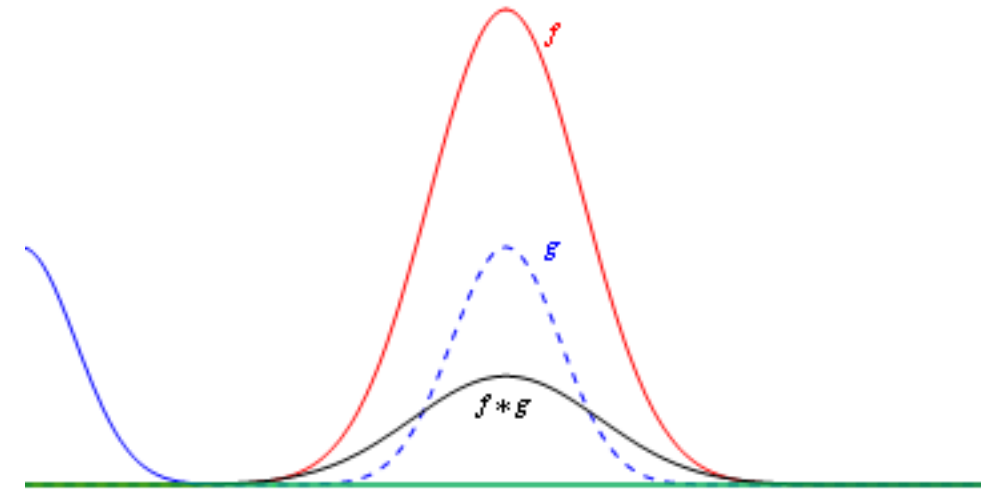
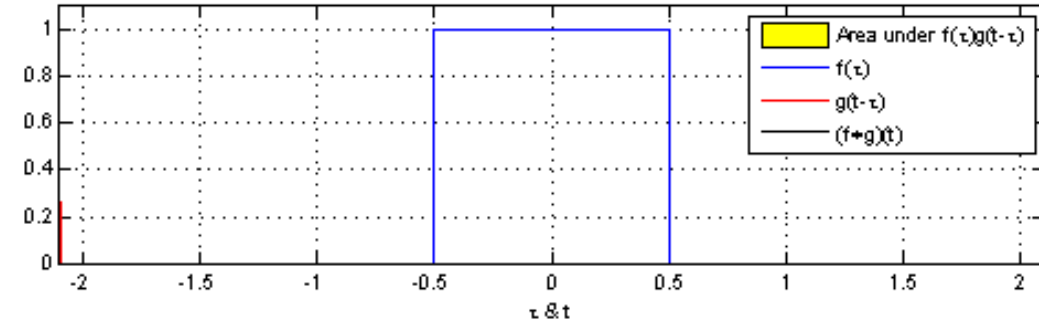
- The convolution is an operation that combines two functions to produce a third function.

$$y(t) = (h * x)(t) := \int_{-\infty}^{\infty} h(\tau)x(t - \tau) d\tau$$

- In words, integral computes the area under the product of $h(\tau)$ and $x(t - \tau)$ as t varies.

- [Demo applet](#)

- **Video:** 3blue1brown / 2023. But what is a convolution? [Link](#)

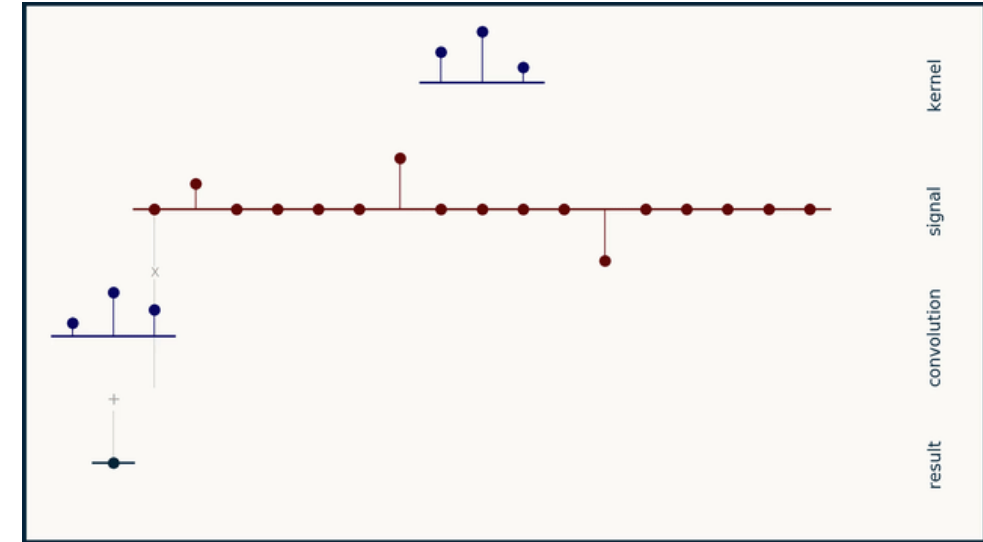


Convolution of discrete-time signals

- There exists also a discrete-time definition for convolution:

$$y[n] = (h * g)[n] = \sum_{m=-\infty}^{\infty} h[m]x[n-m]$$

- In the animation right is $x[n]$ the input signal, $h[n]$ the kernel, and $y[n]$ the output signal.



Naive implementation of a discrete-time convolution

```
# Compute the lengths of our input and filter
N = len(x)
K = len(h)

# Allocate an output buffer
y = np.zeros(N)

# Iterate over delay values
for k in range(K):

    # For each delay value, iterate over all sample indices
    for n in range(N):

        if n >= k:
            # No contribution from samples n < k
            y[n] += h[k] * x[n-k]
```

- [scipy.signal.convolve\(\)](#) is a fast alternative
 - Makes use of FFT!
 - Question: Why again?

Convolution and Fourier transform

- **Result:** The Fourier transform simplifies the convolution operation to a simple multiplication in the frequency domain!

$$\begin{aligned} y(t) &= (h * x)(t) & \Leftrightarrow & Y(\omega) = H(\omega) \cdot X(\omega) \\ y[n] &= (h * x)[n] & \Leftrightarrow & Y[m] = H[m] \cdot X[m] \end{aligned}$$

- Computing the convolution in the frequency domain is very easy!
- We can efficiently recover the time-domain signal via inverse FFT
- Recommended procedure (for discrete signals):
 - Pad $x[n]$ and $h[n]$ to the same length (if necessary)
 - Use the FFT to compute DTFs $X[m]$ and $H[m]$
 - Multiply $X[m]$ and $H[m]$ elementwise
 - Compute the inverse DFT to recover $y[n]$

Signal filtering

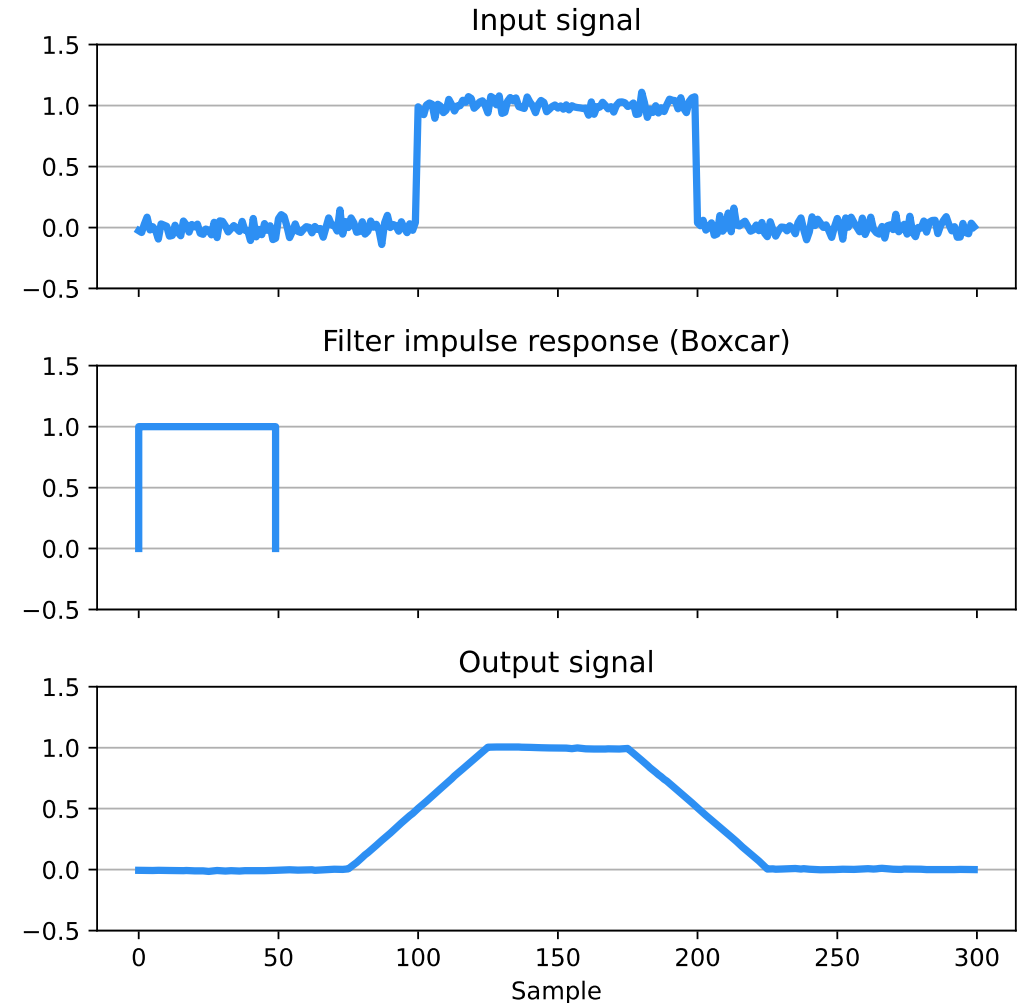
First concepts

Example: Signal smoothing using convolution

- Input:
 - Noisy signal $x(t)$
 - A filter window $h(t)$
- Output
 - Filtered signal $y(t)$
- Solution:

```
import numpy as np
from scipy import signal

noise = 0.05
width = 50
win = signal.windows.boxcar(width)
x = np.repeat([0., 1., 0.], 100)
x += np.random.normal(0, noise, x.shape)
y = signal.convolve(x, win, mode='same') / sum(win)
```



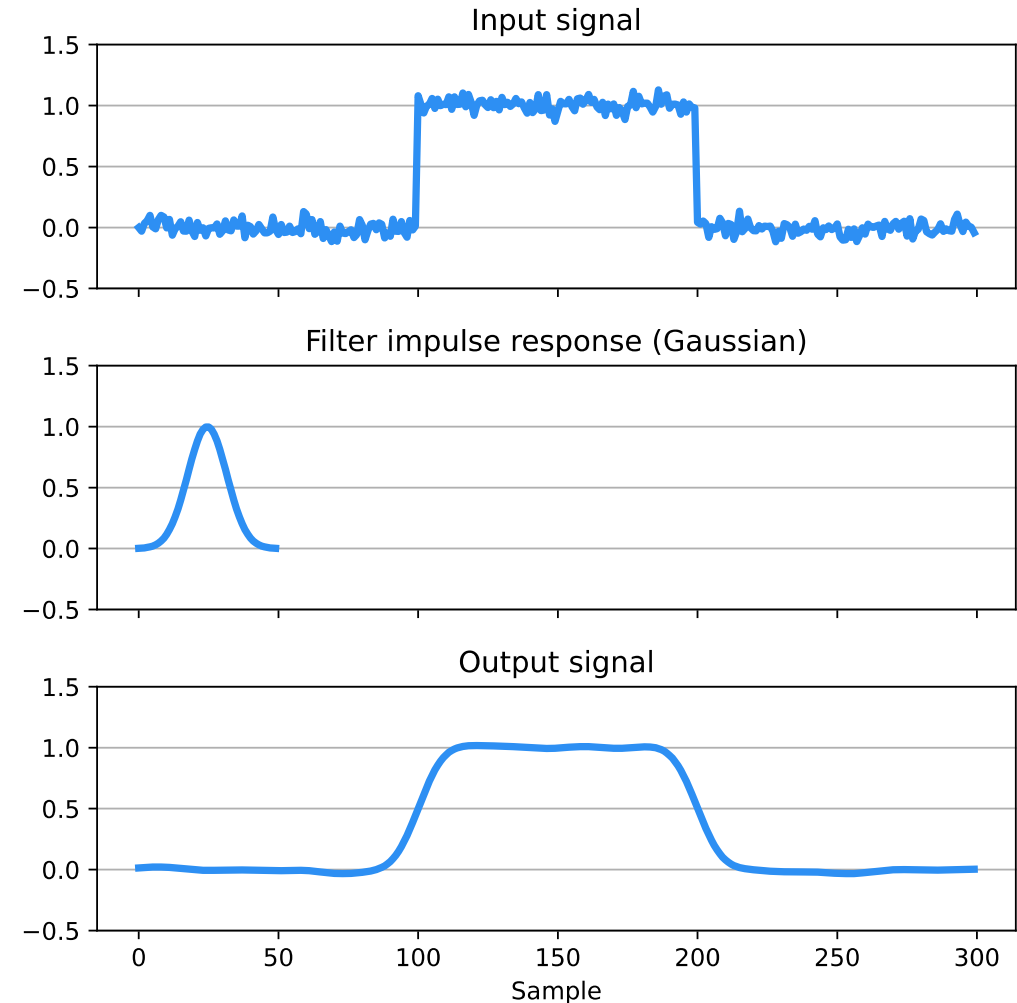
Example: Signal smoothing using convolution

■ Observations:

- We sacrifice some of the signal, but
- We can remove the noise.

```
import numpy as np
from scipy import signal

noise = 0.05
width = 50
win = signal.windows.gaussian(width, std=7)
x = np.repeat([0., 1., 0.], 100)
x += np.random.normal(0, noise, x.shape)
y = signal.convolve(x, win, mode='same') / sum(win)
```



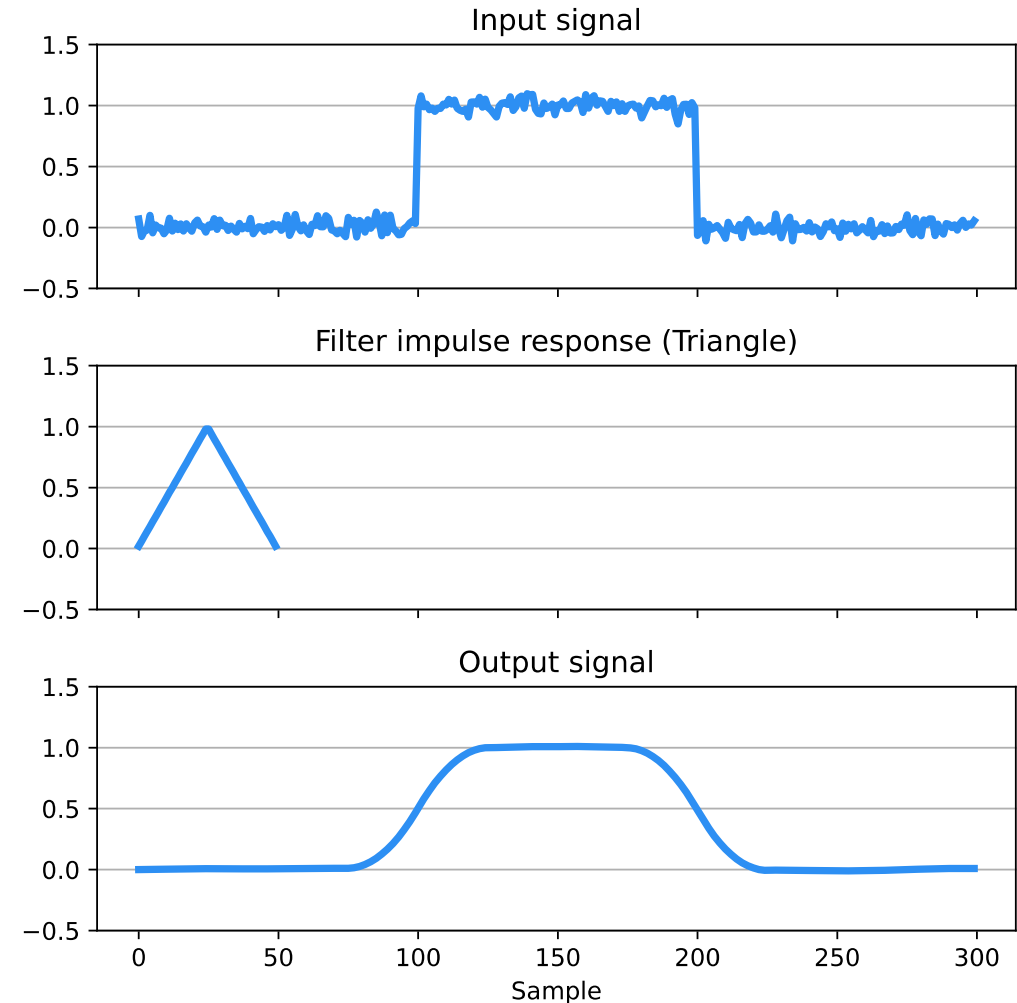
Example: Signal smoothing using convolution

■ Many different window types are possible:

- Box
- Triangle
- Blackman
- Hamming
- Hann
- Parzen
- Exponential
- Tukey
- Lanczos
- ...

```
import numpy as np
from scipy import signal

noise = 0.05
width = 50
win = signal.windows.triang(width)
x = np.repeat([0., 1., 0.], 100)
x += np.random.normal(0, noise, x.shape)
y = signal.convolve(x, win, mode='same') / sum(win)
```

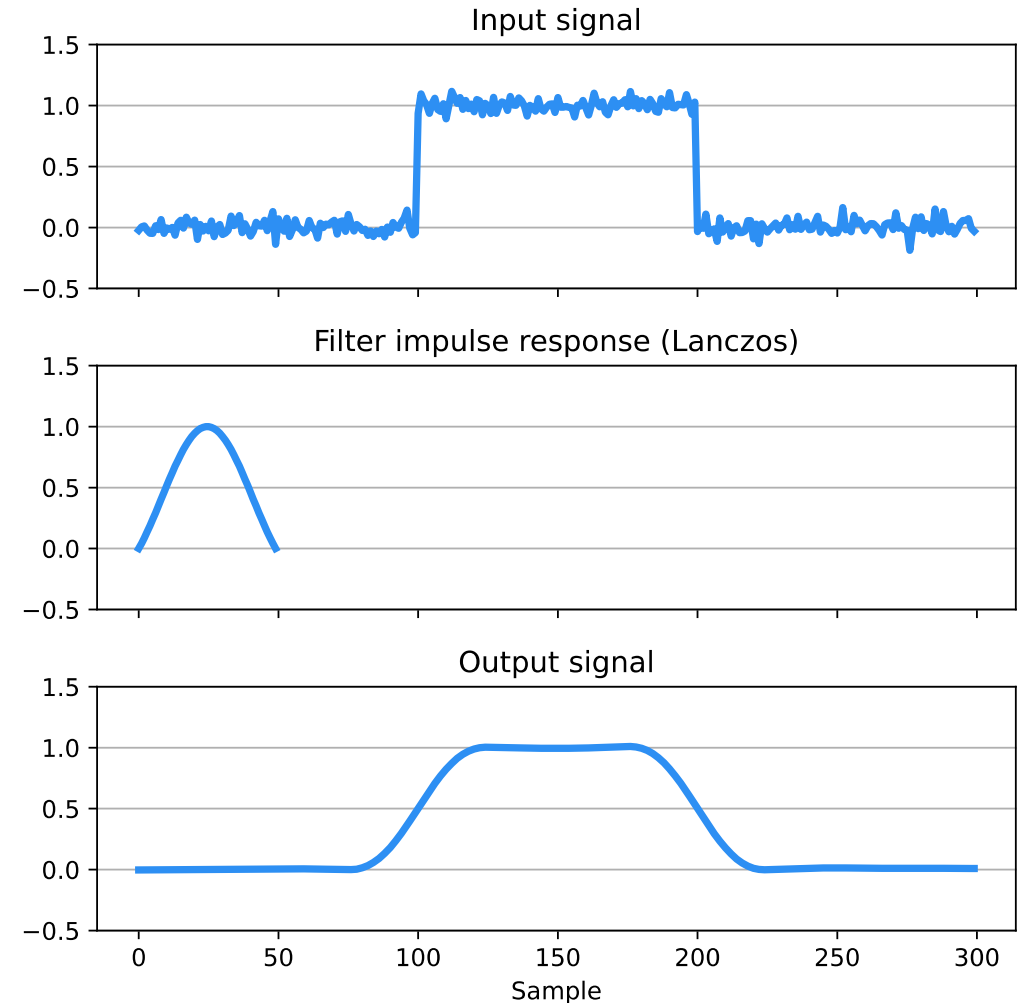


Example: Signal smoothing using convolution

- What type of smoothing window should be used? Which parameters to select?
 - A typical filter design question!
 - Requires an understanding of signal characteristics
 - Spectral analysis of signal and filter!

```
import numpy as np
from scipy import signal

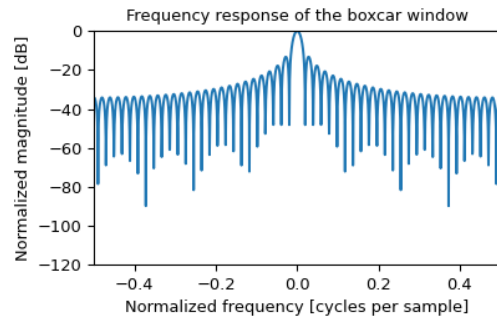
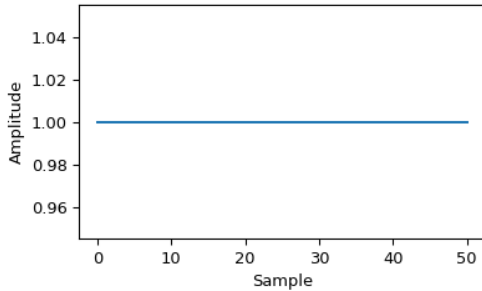
noise = 0.05
width = 50
win = signal.windows.lanczos(width)
x = np.repeat([0., 1., 0.], 100)
x += np.random.normal(0, noise, x.shape)
y = signal.convolve(x, win, mode='same') / sum(win)
```



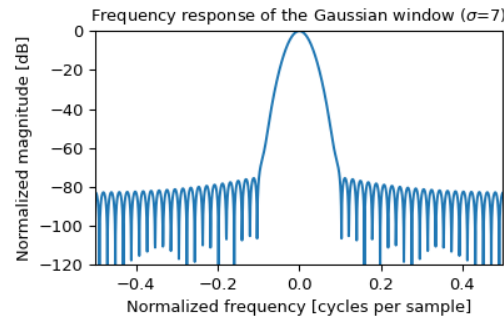
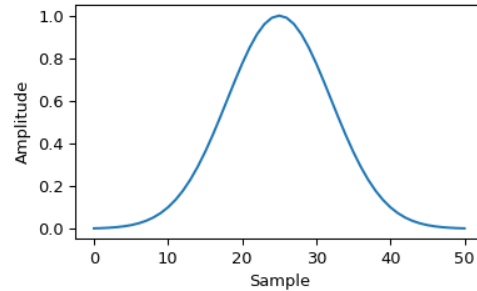
Filters have properties in the time- and frequency domain

Comparison of different window filters. Different window functions lead to different time–frequency trade-offs. Narrow windows preserve detail, while smooth windows reduce spectral artifacts.

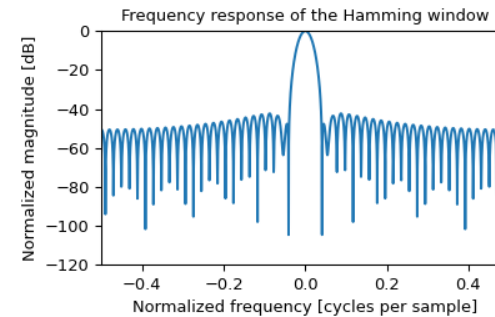
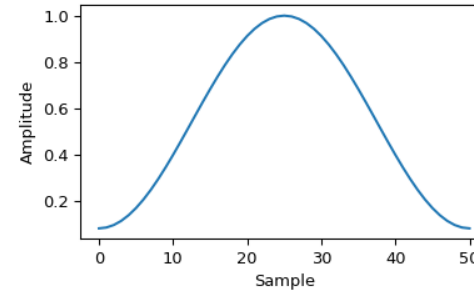
Boxcar window



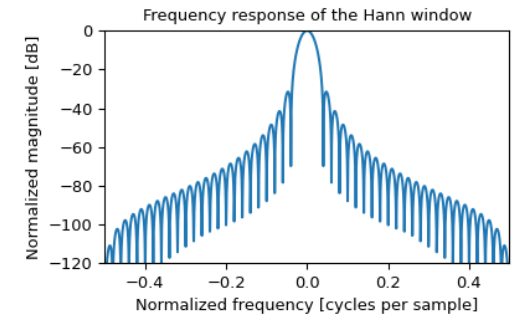
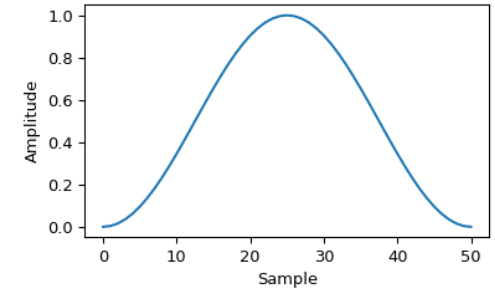
Gaussian window ($\sigma=7$)



Hamming window



Hann window



$$h_{box}[n] = 1$$

$$h_{gauss}[n] = e^{\frac{1}{2}\left(\frac{n}{\sigma}\right)^2}$$

$$h_{lanc}[n] = \text{sinc}_{\pi}\left(\frac{2n}{M-1} - 1\right) \quad h_{hann}[n] = 0.5 - 0.5 \cos\left(\frac{2\pi n}{M-1}\right)$$

$$w_{box}[n] = \begin{cases} 1, & 0 \leq n < N \\ 0, & \text{otherwise} \end{cases}$$

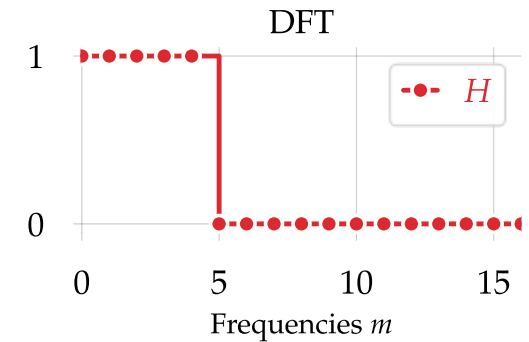
What is the theoretically best low-pass filter?

- Alternative approach: start in the frequency domain
- **Goal:** Get rid of all frequencies above some cut-off f_c
- **Idea:** Let's define the ideal low-pass filter:

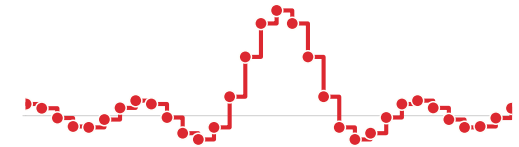
$$H[m] = \begin{cases} 0, & \text{if } f_m > f_c \\ 1, & \text{if } f_m \leq f_c \end{cases}$$

We then can apply the filter in the frequency domain by multiplying it with the signal $X[m]$ and recover the resulting signal $y[n]$ in the time-domain.

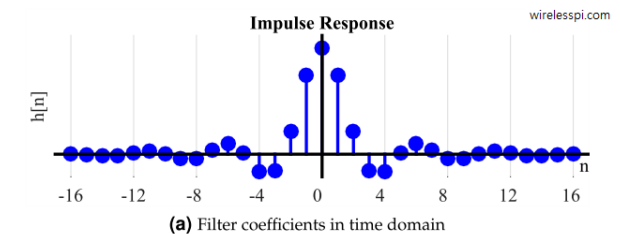
- Such a filter is called **ideal low-pass filter**, or **brick-wall filter**.



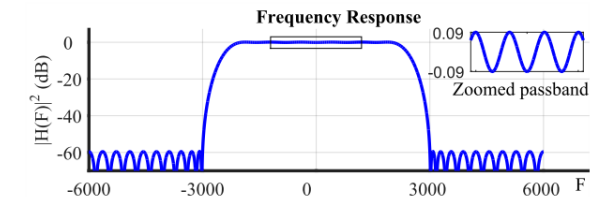
$H[m]$: The ideal low-pass filter in the frequency-domain



$h[n]$: Filter representation in the time-domain (so-called impulse response)



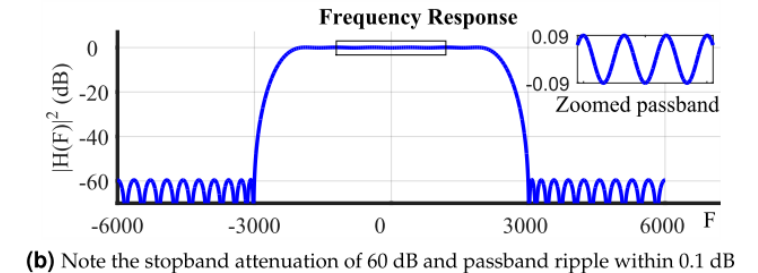
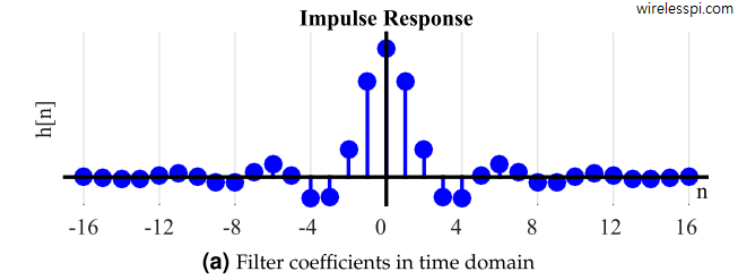
(a) Filter coefficients in time domain



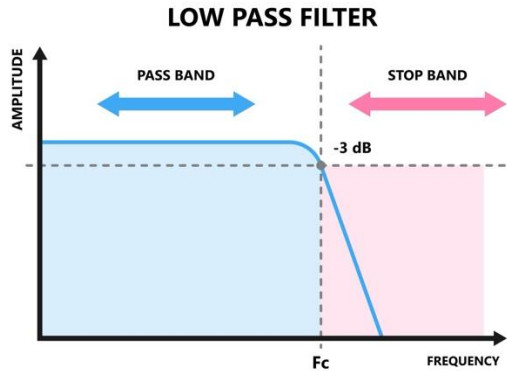
(b) Note the stopband attenuation of 60 dB and passband ripple within 0.1 dB

Problems with the ideal low-pass filter

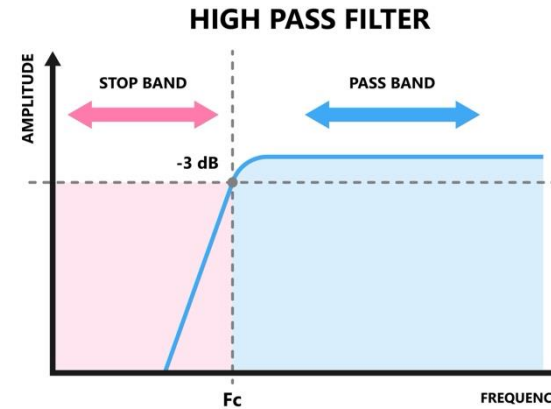
- Has an infinite representation in the time domain
- Cannot be handled properly with DTF, as the filter lengths are limited
- Observations:
 - We already saw that it is difficult to represent sharp edges in the time-domain using frequency components (we require infinitely high frequencies)
 - The same holds true in the frequency domain: The sharper the filter is in the frequency domain, the more time-domain samples are necessary.
 - Note that the filter requires infinitely many samples at $t < 0$! The ideal filter therefore is a non-causal filter.
- See [here](#) for an illustrative demonstration.



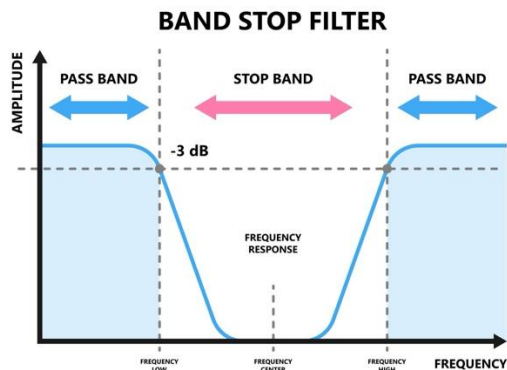
Overview: Types of filters



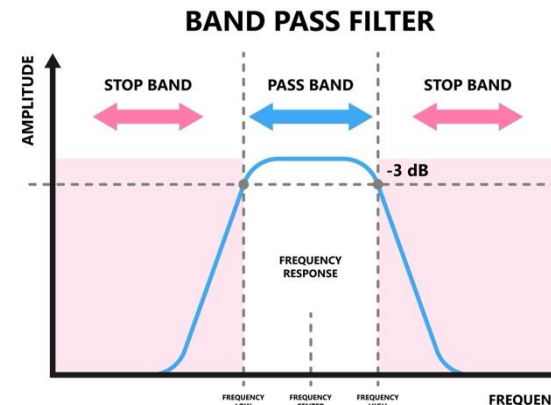
- Allows frequencies below a cutoff frequency to pass through while attenuating higher frequencies.
- Used to remove high-frequency noise, smooth signals, or extract low-frequency components.



- Allows frequencies above a cutoff frequency to pass through while attenuating lower frequencies.
- Used to remove low-frequency noise, extract high-frequency components, or emphasize high-frequency features.



- Attenuates frequencies within a specific range, known as the stopband, while allowing frequencies outside this range to pass through.
- Used to remove unwanted interference or noise within a certain frequency range while preserving the rest of the signal.



- Allows a specific range of frequencies, known as the passband, to pass through while attenuating frequencies outside this range.
- Used to isolate or extract signals within a certain frequency band while rejecting others.